

[기업연계] 언리얼 엔진 게임 개발자 부트캠프 8기

Technical documentation

# 브롤스타즈 TPS

언리얼 엔진을 활용한 탐다운 슈팅 게임

"브롤스타즈" 재해석 게임

김민영

[zxc9876zxc@gmail.com](mailto:zxc9876zxc@gmail.com)





# 프로젝트 개요



# 목차

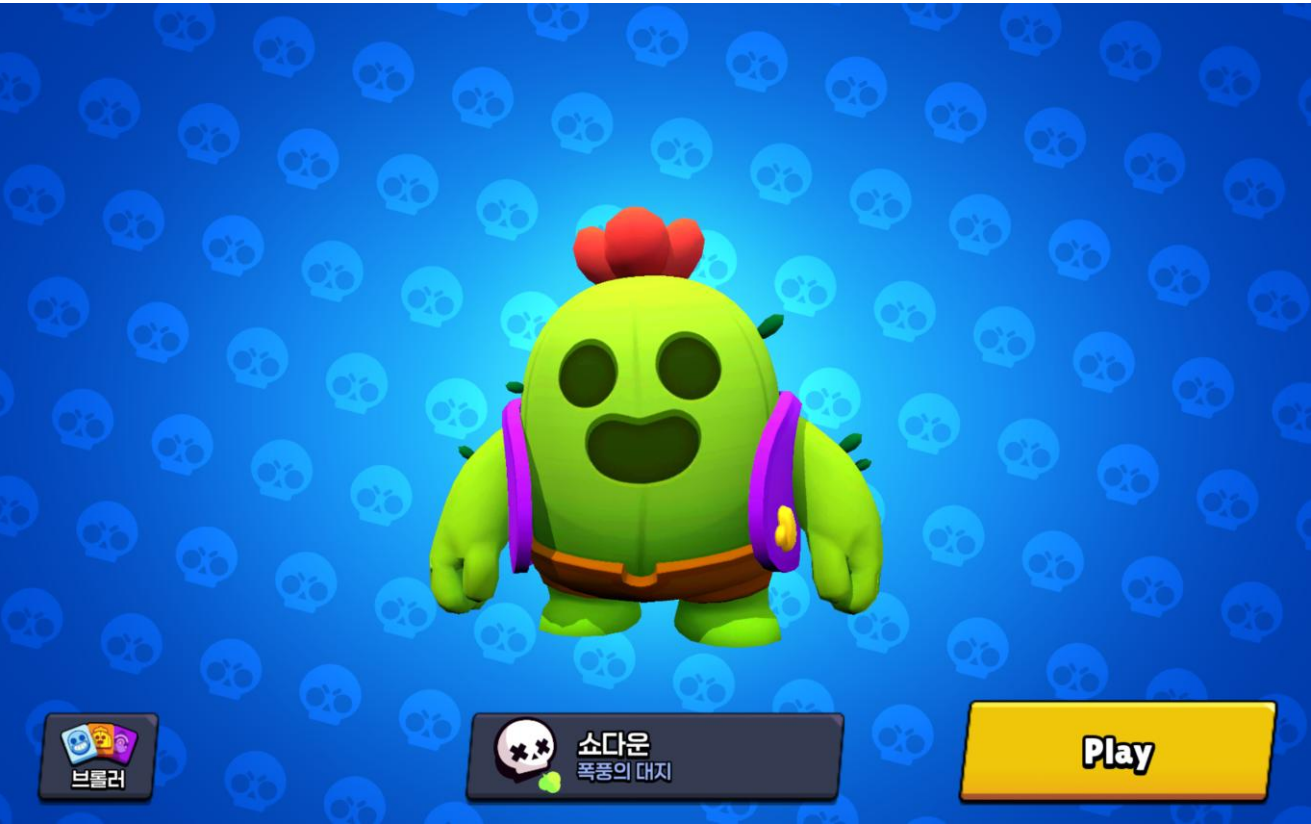
---

|    |          |                                                                                                                                           |
|----|----------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 01 | 프로젝트 개요  | 게임 소개 ( <a href="#">04</a> ) · 왜 <b>TPS</b> 인가 ( <a href="#">05</a> ) · 게임 흐름도 ( <a href="#">06</a> )                                     |
| 02 | 주요 특징    | 캐주얼 그래픽 ( <a href="#">08 ~ 09</a> ) · 자동 조준 ( <a href="#">10</a> )                                                                        |
| 03 | 핵심 기능 구현 | <b>GAS</b> ( <a href="#">12 ~ 13</a> ) · 동적 <b>AI</b> 행동 트리 주입 ( <a href="#">14 ~ 15</a> ) · <b>IK Retargeting</b> ( <a href="#">16</a> ) |
| 04 | 프로젝트 회고  | 개선점 + 느낀 점 ( <a href="#">18</a> )                                                                                                         |

---



게임 소개

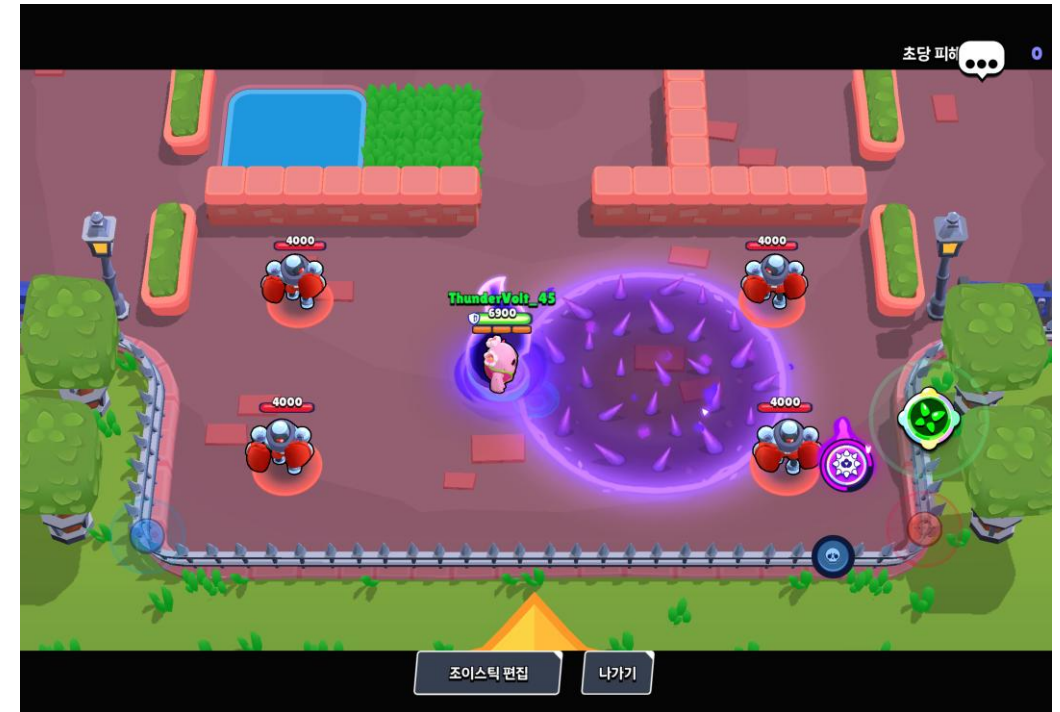
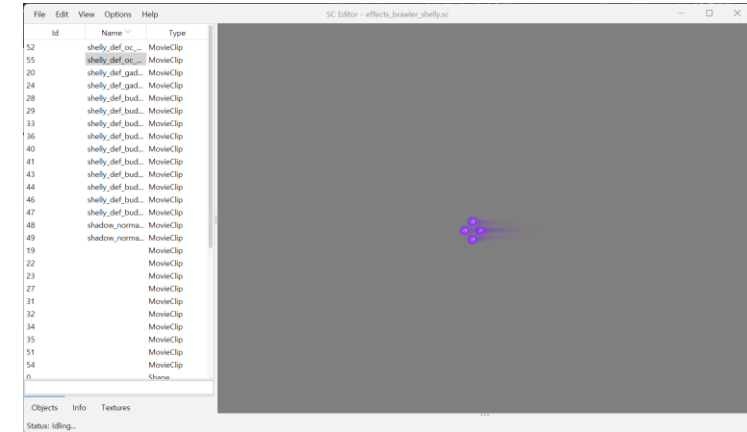


|         |                                                                                                                                                                                                                                                                   |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 게임명     | 브롤스타즈 <b>TPS</b>                                                                                                                                                                                                                                                  |
| 개발 환경   | <b>Unreal Engine 5.7 + Rider</b>                                                                                                                                                                                                                                  |
| 장르      | <b>TPS</b>                                                                                                                                                                                                                                                        |
| 플랫폼     | <b>PC (Windows)</b>                                                                                                                                                                                                                                               |
| 개발 기간   | 2026.01.19 ~ 2026.02.19 (4주)                                                                                                                                                                                                                                      |
| 인원      | 1인                                                                                                                                                                                                                                                                |
| 프로젝트 링크 |    |



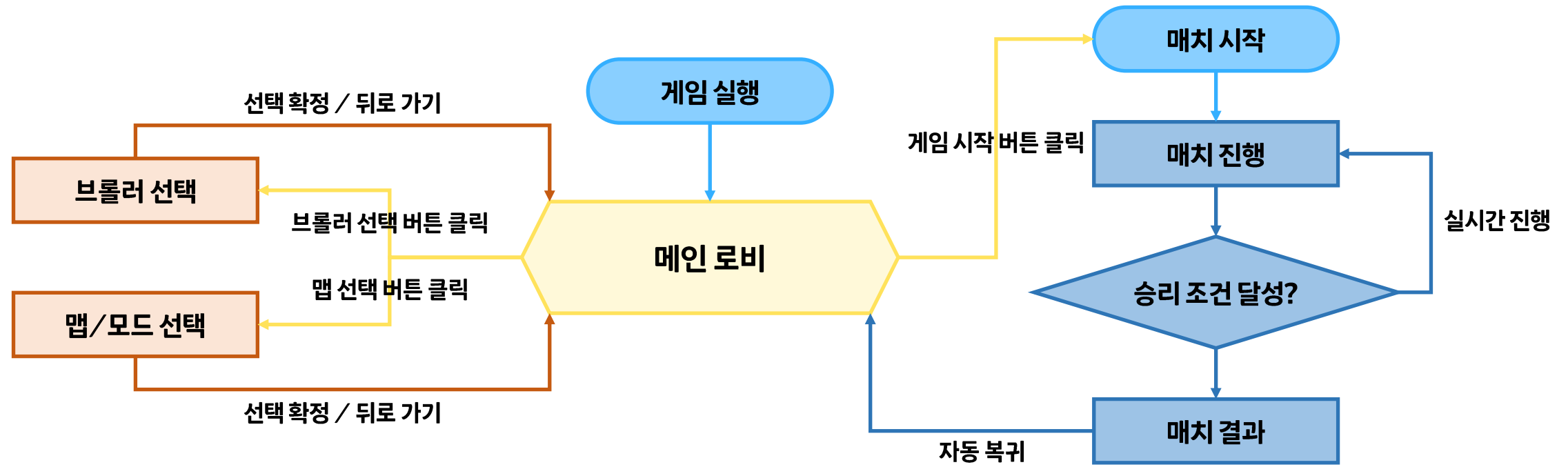
## 왜 TPS인가?

- 원작 브롤스타즈를 구성하는 에셋은 **3D 모델링**과 **2D 벡터 그래픽**으로 구성되어 있음
- 특히 발사체나 **VFX** 같은 효과의 거의 대부분이 **2D 벡터 그래픽**으로 만들어져 언리얼 엔진과 맞지 않음
- 대부분의 에셋이 탑다운 시점을 기준으로 만들어져서 에셋의 유용이 곤란함
- 어차피 새로 만들어 써야 한다면...? 그냥 **TPS** 장르로 재해석해보자





# 게임 흐름도





# 주요 특징

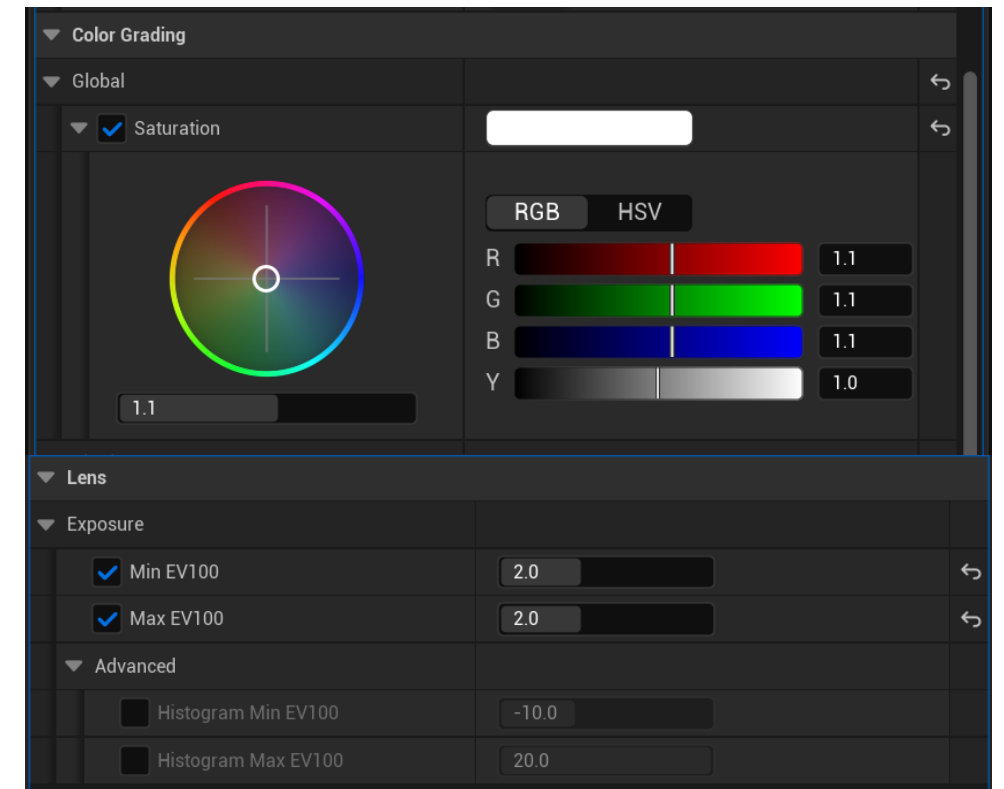
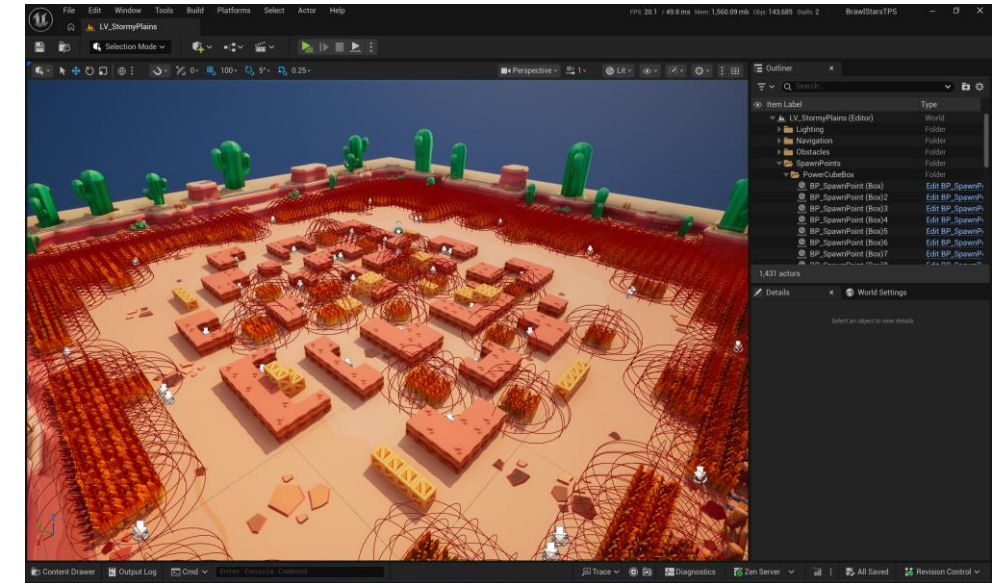
## 02 · 주요 특징



# 캐주얼 그래픽

포스트 프로세싱, 머테리얼을 활용한 캐주얼 그래픽 구현

- **PostProcessVolume**
  - 렌더링이 완료된 화면에 최종적으로 필터 효과를 적용하여 색보정(**Color Grading**), 톤매핑, 블룸(**Bloom**), 노출(**Exposure**) 등을 제어하는 기능
- **Saturation** 값을 높여 화면 전체의 채도를 늘림
- **EV100의 Min, Max** 값을 일치시켜 자동 노출 조정 기능 비활성화, 화면 전체의 밝기가 개발자의 의도대로 제어되도록 함

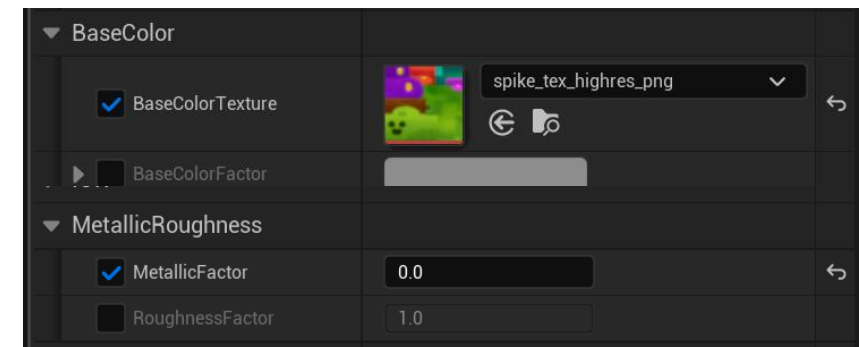
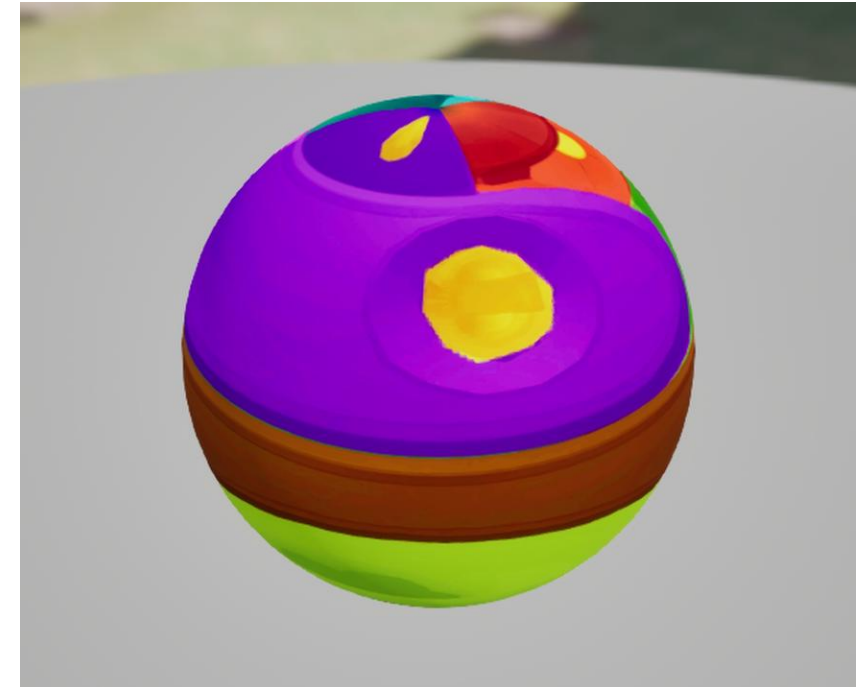




## 캐주얼 그래픽

포스트 프로세싱, 재터리를 활용한 캐주얼 그래픽 구현

- **Material**
  - 오브젝트 표면에 적용되어 컬러, 질감, 반사, 투명도 등 물리적 특성을 정의하는 "표면 페인트"
- **Roughness** 값을 최대로, **Metallic** 값을 최소로 설정
- 캐릭터가 받는 빛을 완벽하게 난반사, 확산시켜 캐릭터의 고유 색을 명확하게 전달하면서 동시에 부드러운 질감을 갖게 함

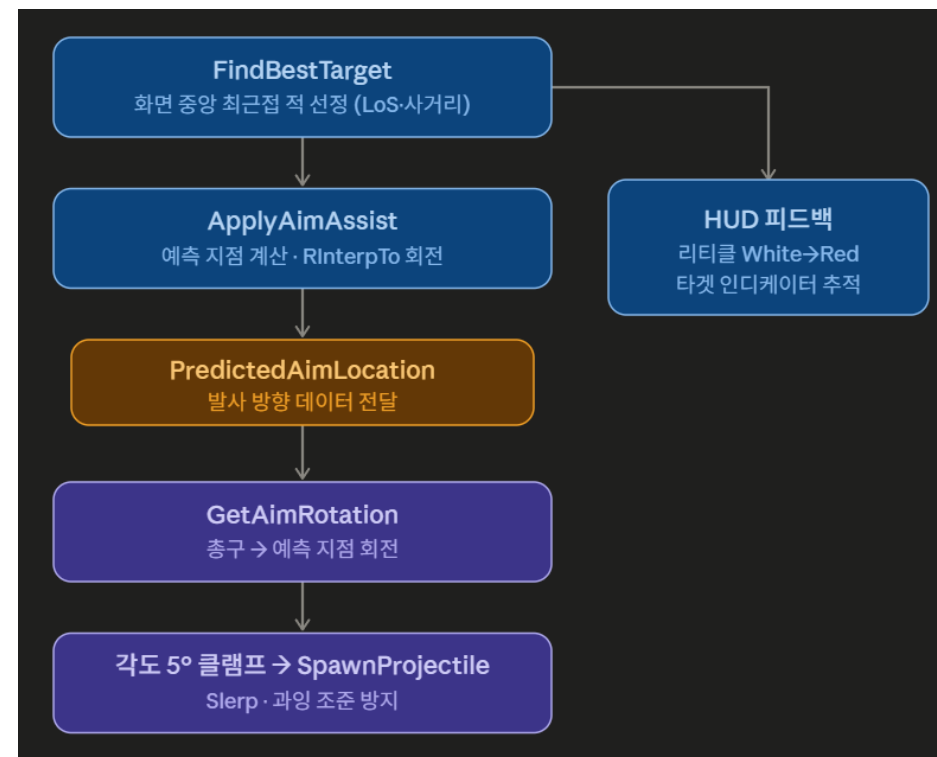




## 자동 조준

### 브롤스타즈의 Auto Aim 기능을 TPS 장르에 맞춰 재해석

- **TPS** 게임에서 브롤스타즈의 캐주얼한 조준감을 재현하고자 도입한 시스템으로 화면 중앙의 적을 자동으로 포착하고, 움직이는 표적의 미래 위치를 예측해 조준을 보정
- 유저가 바라보는 화면의 중심을 기준으로 **SphereTrace** 검사를 수행해 에임과 가장 가까운 타겟 선정
- 표적의 현재 속도를 고려해 적의 미래 위치를 예측하고 발사체의 속도에 따른 탄착 시간을 고려해 에임을 오버 슈트
  - (탄착시간 = 거리 / 발사체속도 → 예측지점 = 표적위치 + 표적속도 × 탄착시간)
- **RInterpTo**로 카메라를 서서히 정렬, 발사 시 최대 5° 이내로 발사체 방향 추가 보정



자동 조준 기능 시연 영상



# 핵심 기능 구현



# GAS

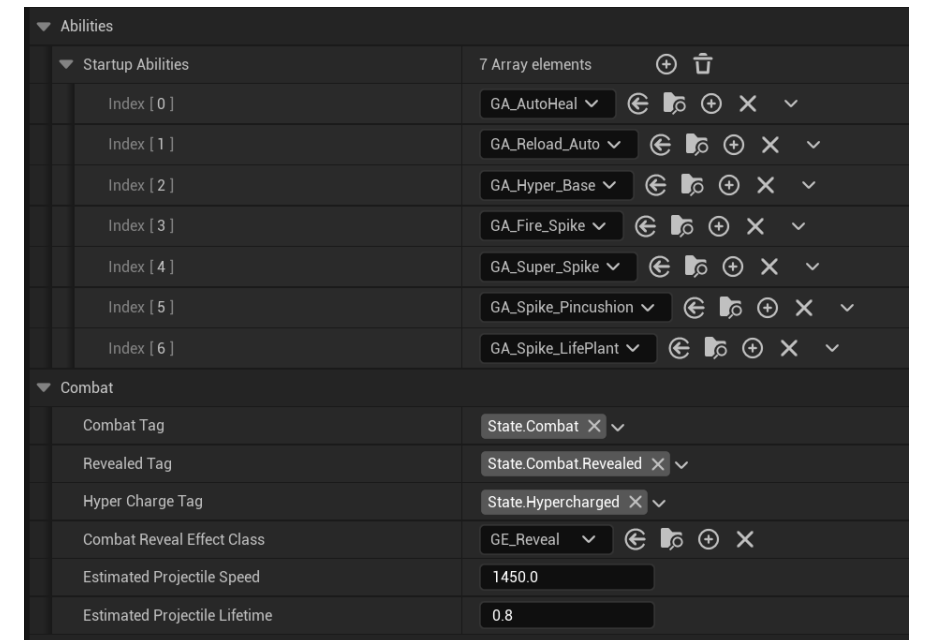
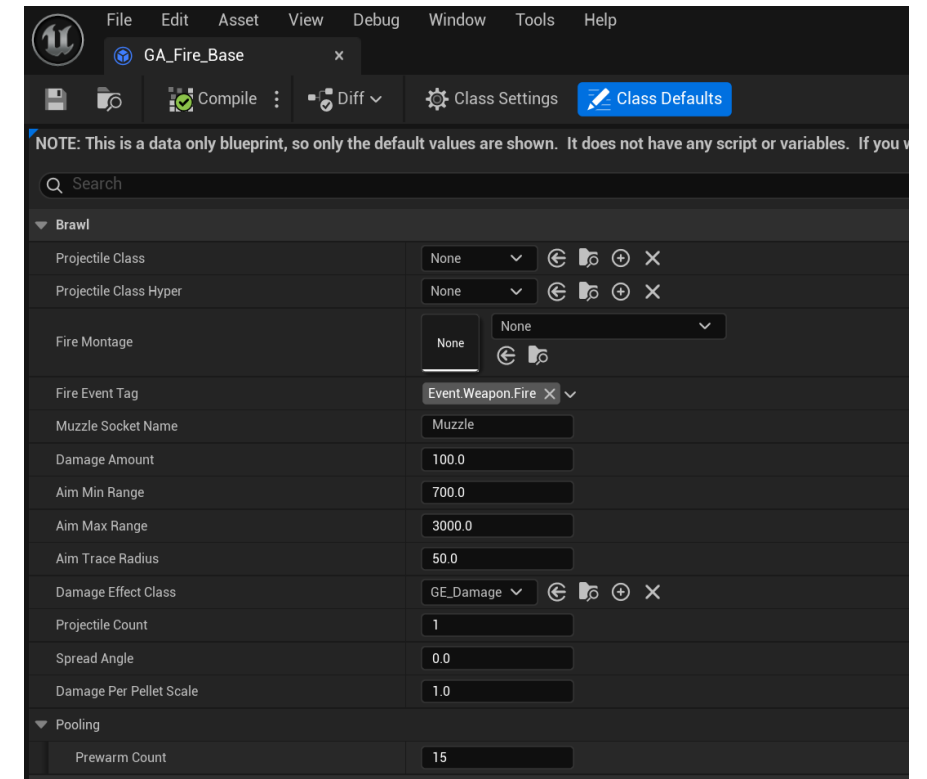
## 복잡한 Ability, Attribute 구축을 지원하는 Framework

### 문제

- 브롤러마다 고유한 평타·궁극기·가젯·하이퍼차지를 가짐 → 캐릭터마다 스킬을 하드코딩하면 **N**개 브롤러 × **M**개 스킬의 조합 폭발
- 데미지/궁극기 충전/방어/버프 등이 서로 얽혀 있어, 캐릭터 코드에 로직을 넣으면 유지보수 불가

### 해결책 = GAS

- Ability** = 독립 객체로 스킬 하나가 곧 하나의 클래스. 캐릭터는 "어떤 **Ability**를 들고 있나"만 알면 됨 → **Ability** 조합만으로 신규 브롤러 생성
- Attribute/Effect** 분리: 데미지·쿨다운·게이지 충전을 **GameplayEffect**라는 데이터로 표현 → 밸런싱을 코드 수정 없이 가능
- GameplayTag**: **State.Combat**, **State.Hypercharged** 같은 상태를 문자열 태그로 관리 → 조건 분기와 **UI** 연동을 느슨하게 결합





# GAS

## 복잡한 Ability, Attribute 구축을 지원하는 Framework

### 구성 — 데이터 주도 · 컴포넌트 조립

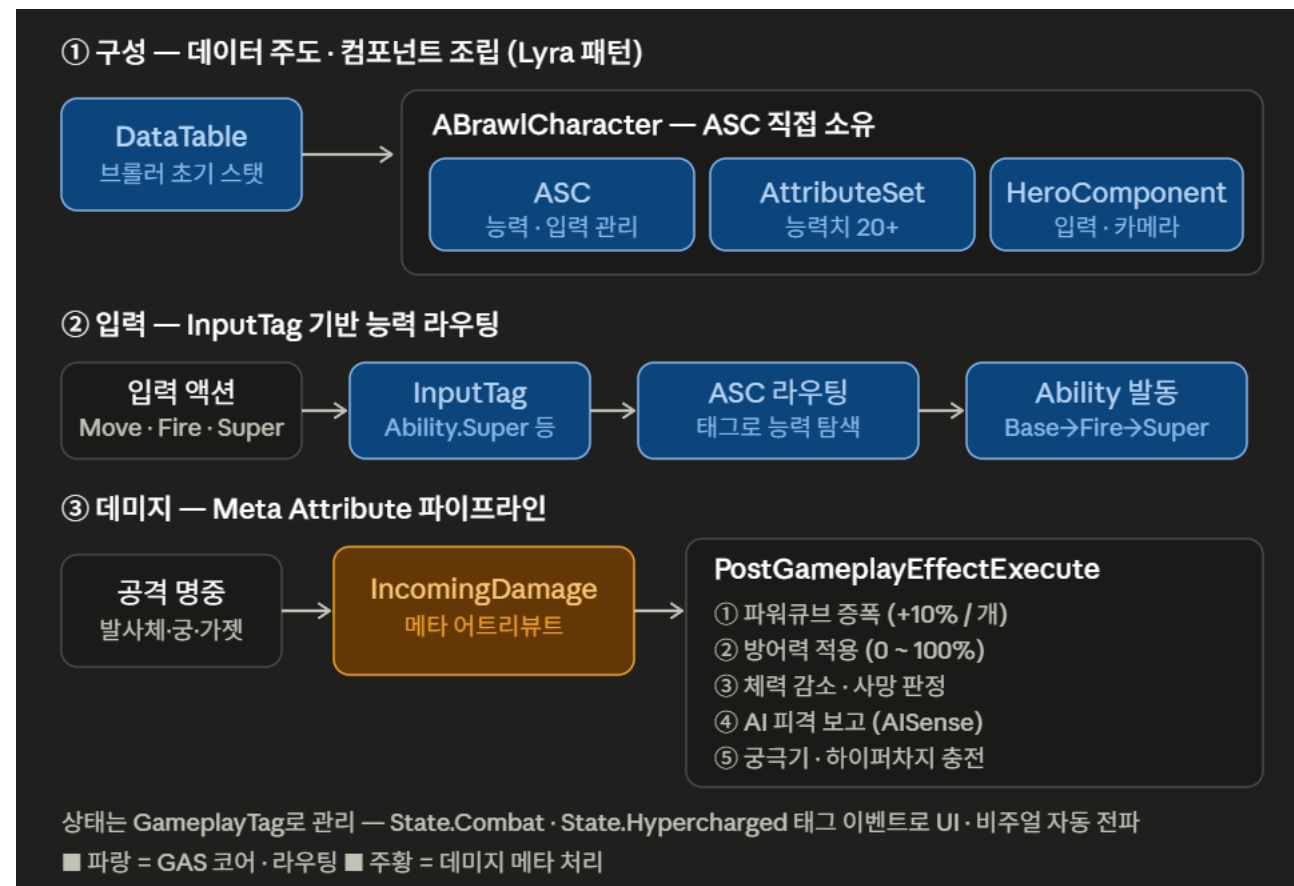
- **ASC·AttributeSet·HeroComponent**를 **ABrawlCharacter**에 조립 (**Epic Games**의 **Lyra**식 **Pawn/Hero** 컴포넌트 패턴)
- 브롤러별 스탯은 **DataTable**에 정의 → 동적으로 주입
- 신규 브롤러 = 데이터 한 줄 + **Ability** 조합 (하드코딩 제거)

### 입력 — InputTag 기반 능력 라우팅

- 입력 → 함수 하드 바인딩 대신 **InputTag.Ability.Super** 같은 태그로 **ASC**가 능력을 탐색·활성화
- 입력과 능력이 완전히 분리 → 키 매핑·신규 스킬 추가가 자유로움
- **Ability** 상속 계층(**BrawlGameplayAbility** → **Fire** → **Super**)으로 공통 로직 재사용

### 데미지 — Meta Attribute 파이프라인

- 모든 데미지는 체력을 직접 깎지 않고 **IncomingDamage**(메타 어트리뷰트) 한 곳을 경유
- 데미지 증폭 → 방어 → 사망 판정 → 게이지 충전을 단일 지점에 집중
- 데미지 판정의 경로 일관성 확보 및 코드 중복 최소화





## 동적 AI 행동 트리 주입

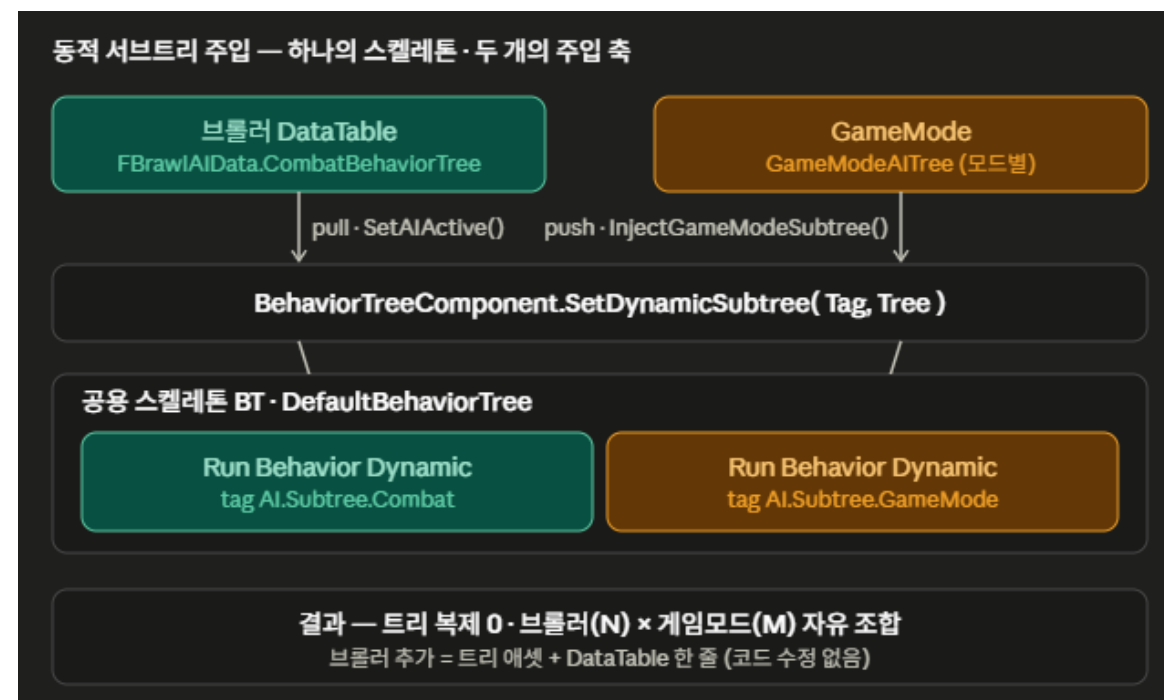
하나의 스켈레톤 트리로 브롤러 N종 \* 게임 모드 M종 대응

### 문제

- 브롤러마다 고유 매커니즘의 공격, 스킬을 가짐 → 하지만 브롤러 수만큼 행동 트리를 통째로 복제하면 유지보수 불가
- 게임 모드마다 모드의 승리 조건에 맞춰 추가 행동이 필요 → 트리를 모드별로 또 분기해야 함

### 해결책 = 서브 행동 트리

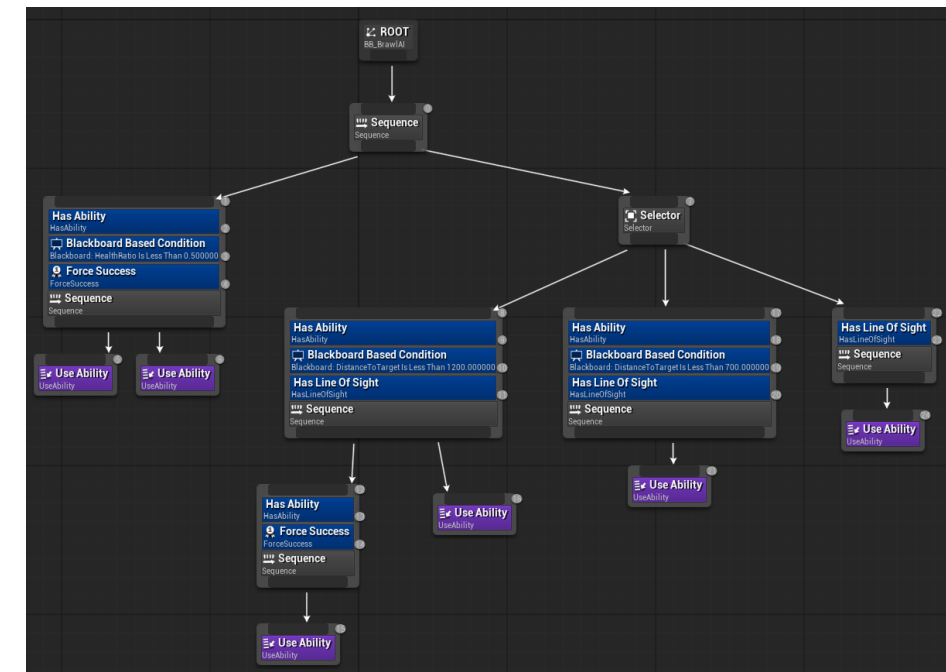
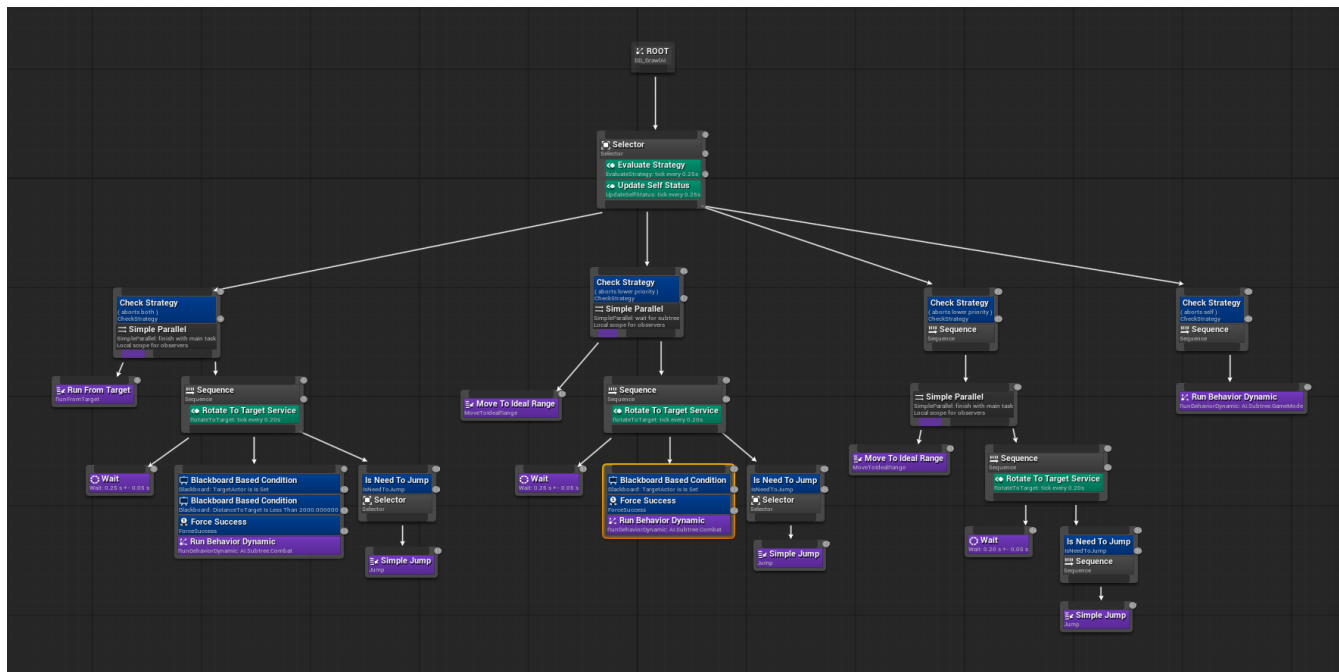
- 모든 브롤러가 공유하는 공통 로직만 스켈레톤 행동 트리로 작성
- 각 브롤러가 갖는 고유 전투 매커니즘에 맞춘 서브 행동 트리과 각 게임 모드의 승리 조건을 수행하는 서브 행동 트리를 런타임에서 동적으로 끼워넣음





# 동적 AI 행동 트리 주입

하나의 스켈레톤 트리로 브롤러 N종 \* 게임 모드 M종 대응

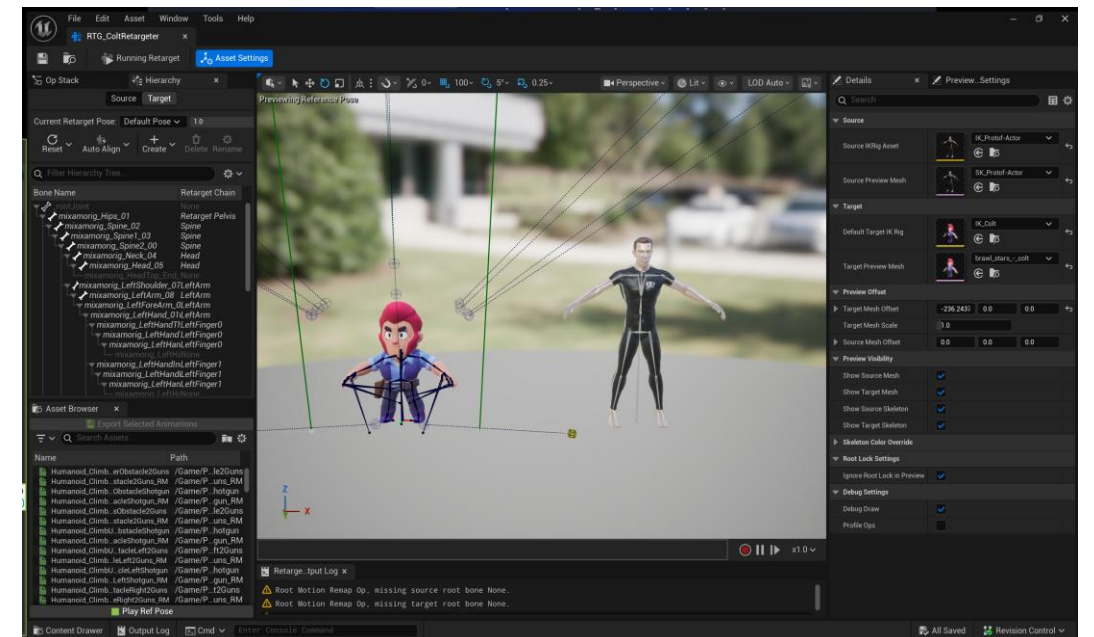
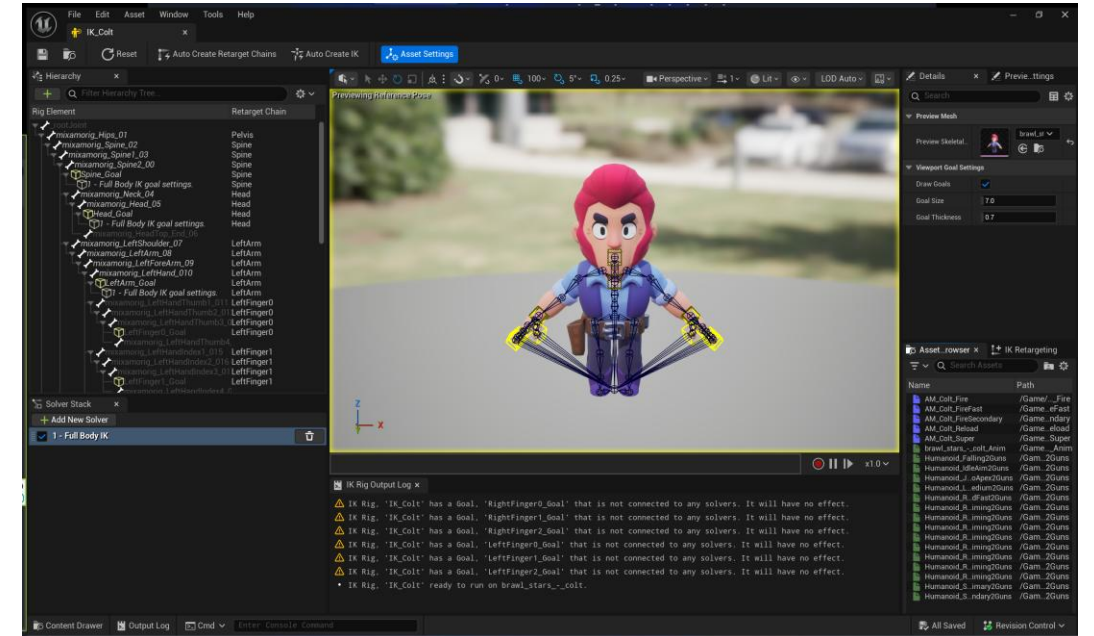


- 모든 브롤러가 공통으로 수행할 로직만 남긴 공용 스켈레톤 행동 트리를 작성
- 스켈레톤 행동 트리 내에서 각 브롤러/게임 모드 별로 로직이 달라져야 하는 부분에 **Tag**가 붙은 빈 노드(**Run Behavior Dynamic**)를 배치
- 런타임에서 **BehaviorTreeComponent.SetDynamicSubtree(Tag, Tree)**로 실제 트리를 그 슬롯에 끼워 넣음

브롤러나 게임 모드가 늘어나더라도 스켈레톤 행동 트리는 그대로, 브롤러와 게임 모드 별 전용 행동 트리만 추가하면 OK

## 같은 스케탈을 쓰지 않는 에셋 간의 애니메이션 강제 변환

- 언리얼 엔진에서는 같은 스켈레탈을 가지는 에셋만 애니메이션이 호환됨
- 하지만 브롤러 모델과 애니메이션 에셋 모두가 언리얼 표준 스켈레탈을 따르지 않는 독자 규격 → **IK Retargeting**으로 해결 가능
- 브롤러 모델과 애니메이션 에셋의 **IK Rig**을 제작하고 각각 대응하는 **Bone**에 **IK Goal** 설정, 이후 만들어진 **IK Rig**을 기반으로 **IK Retargeter**를 생성
- **IK Retargeter**를 통해 애니메이션 에셋을 각 모델에 맞게 변환하면 호환되지 않는 애니메이션 에셋도 사용 가능





# 프로젝트 회고



## 개선점 + 느낀 점

### 개선점

- **Aim Offset** 기능을 통한 조준 방향 총구 정렬 구현 필요
- 브롤러 **AI**의 추가 고도화 필요

### 느낀 점

- 언리얼 엔진 프로젝트를 진행하면서 언리얼 엔진 이해도 향상
- **GAS** 적용을 통한 최신 기법 이해도 향상

" 처음으로 수행했던 언리얼 엔진 **C++** 프로젝트로서 많은 시행 착오가 있었지만  
언리얼 엔진에 대한 이해도와 **GAS**와 같은 최신 기술에 대한 이해도를 높일 수 있던 아주 소중한 시간이었습니다. "



감사합니다